

SPI Bus Communication Experiment

SPI Bus Communication Experiment

1. SPI Communication Overview
2. Core Concepts
 - 2.1 SPI Four-Wire System
 - 2.2 SPI Modes and Polarity
 - 2.3 Core API
3. Hardware Dependencies
 - 3.1 Pin Mapping
 - 3.2 Master-Slave Communication Wiring (All Four Wires + GND)
4. Project Architecture and Flow
5. Source Code Details
 - 5.1 Pin Configuration
 - 5.2 SPI Initialization
 - 5.3 Full-Duplex Transfer
 - 5.4 Main Loop
6. Operation Procedure
 - 6.1 Arduino IDE Environment Setup
 - 6.2 Flashing and Running
 - 6.3 Arduino Slave Code (slave.ino)
 - 6.4 SPI Full-Duplex Timing Description
 - 6.5 Master Serial Output Example
 - 6.6 Slave Serial Output Example
 - 6.7 Verification Method
7. Common Issues

Dual Environment Description: This experiment uses **MicroPython + Thonny** as the host and **Arduino + Arduino IDE** as the slave. This is because MicroPython's `machine.SPI` module only supports SPI master mode and lacks a slave API—the slave must be implemented using the ESP-IDF `spi_slave` driver. Therefore, this tutorial uses the Arduino-ESP32 framework (based on ESP-IDF) to write the slave code.

1. SPI Communication Overview

SPI (Serial Peripheral Interface) is a high-speed full-duplex synchronous serial bus. Standard SPI uses **four wires**: SCK (clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), CS (chip select). In this experiment, the ESP32-S3 acts as the SPI master and performs master-slave communication with the slave.

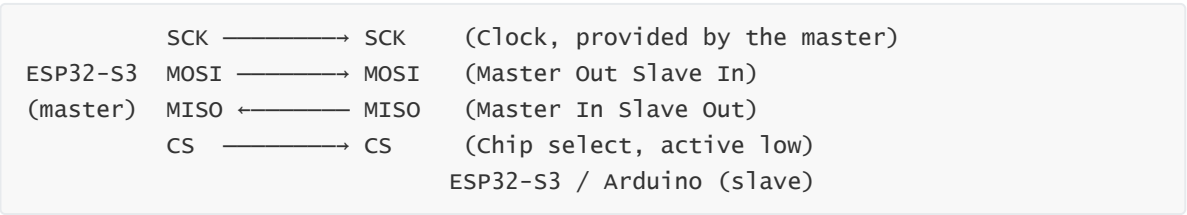
Typical application scenarios:

Application	Example
Display driver	TFT/OLED screens (ST7789, ILI9341)
Sensors	BME280, MPU9250, MAX31865
Storage	SD card, external Flash (W25Qxx)
High-speed communication	Large data transfer between MCUs

SPI is much faster than I2C and UART, suitable for speed-sensitive scenarios.

2. Core Concepts

2.1 SPI Four-Wire System



- **SCK:** The master generates the clock; each clock pulse transfers 1 bit
- **MOSI:** Master sends → slave receives
- **MISO:** Slave sends → master receives
- **CS (NSS):** Pull low to select the slave, pull high to release

SPI is full-duplex — it can receive while sending; throughput = clock frequency.

2.2 SPI Modes and Polarity

SPI has 4 modes, determined by **CPOL (clock polarity)** and **CPHA (clock phase)**:

Mode	CPOL	CPHA	Idle clock	Sampling edge
0	0	0	Low	Rising edge
1	0	1	Low	Falling edge
2	1	0	High	Falling edge
3	1	1	High	Rising edge

This experiment uses **Mode 0** (most common), `polarity=0, phase=0`.

2.3 Core API

```
from machine import Pin, SPI

spi = SPI(1,
          baudrate=115200,      # Clock frequency 115.2kHz
          polarity=0, phase=0,  # SPI mode 0
          sck=Pin(4), mosi=Pin(5), miso=Pin(6))

cs = Pin(10, Pin.OUT, value=1) # CS initially high (inactive)

cs.off()                        # Pull CS low → select the slave
spi.write_readinto(tx_buf, rx_buf) # Simultaneous TX & RX
cs.on()                        # Pull CS high → release
```

3. Hardware Dependencies

3.1 Pin Mapping

SPI four-wire system — SCK, MOSI, MISO, CS:

GPIO	Peripheral	Direction	Description	Function
4	SPI1 SCK	Output	Clock	Clock signal, generated by the master
5	SPI1 MOSI	Output	Master Out Slave In	Master sends, slave receives
6	SPI1 MISO	Input	Master In Slave Out	Slave sends, master receives
10	SPI1 CS	Output	Chip Select (active low)	Chip select (active low), selects the target slave

3.2 Master-Slave Communication Wiring (All Four Wires + GND)

```
Master GPIO4 (SCK)  → Slave GPIO4 (SCK)
Master GPIO5 (MOSI) → Slave GPIO5 (MOSI)
Master GPIO6 (MISO) ← Slave GPIO6 (MISO)
Master GPIO10 (CS)  → Slave GPIO10 (CS)
GND                 — GND
```

4. Project Architecture and Flow

```

Script execution (single-file .py)
|
├─ init(): Initialize SPI1 (115.2kHz, MODE0) + CS(GPIO2, high)
|
└─ main() main loop:
    ├─ transfer(spi, cs, tx, rx):
    |   ├─ cs.off()           Pull the chip select low, select the
slave
    |   │   └─ spi.write_readinto()    Full-duplex transfer
    |   └─ cs.on()            Pull the chip select high, release the
slave
    └─ Parse the received data (rx → string)
        ├─ print TX / RX
        └─ sleep(2s)

```

5. Source Code Details

Complete source code: [5.Source Code](#) → [MicroPython](#) → [3.Serial Communication and Storage System](#) → [3. SPI_Master](#) → [master.py](#)

5.1 Pin Configuration

```

# Pins
MST_SCK = 4      # SPI clock pin
MST_MOSI = 5     # MOSI pin
MST_MISO = 6     # MISO pin
MST_CS = 10     # CS pin

BUF_SIZE = 64    # TX/RX buffer size
BAUDRATE = 115200 # SPI clock frequency

```

5.2 SPI Initialization

```

def init():
    spi = SPI(1,
               baudrate=BAUDRATE,
               polarity=0, phase=0,      # SPI mode 0
               sck=Pin(MST_SCK),
               mosi=Pin(MST_MOSI),
               miso=Pin(MST_MISO))
    cs = Pin(MST_CS, Pin.OUT, value=1)  # CS initially high (inactive)
    print("[MASTER] SPI(1) initialized")
    print("      SCK=%d MOSI=%d MISO=%d CS=%d"
          % (MST_SCK, MST_MOSI, MST_MISO, MST_CS))
    return spi, cs

```

5.3 Full-Duplex Transfer

```
def transfer(spi, cs, tx_buf, rx_buf):
    """Single SPI transfer"""
    for i in range(BUF_SIZE):
        rx_buf[i] = 0

    cs.off()                # CS low, select slave
    spi.write_readinto(tx_buf, rx_buf) # Simultaneous TX & RX
    cs.on()                 # CS high, release slave
```

`write_readinto` is an efficient full-duplex SPI API in MicroPython — it sends the `tx_buf` data and fills `rx_buf` with the received data at the same time, without needing two separate write-then-read passes.

5.4 Main Loop

```
msg = b"Hello from MASTER"
tx = bytearray(BUF_SIZE)
tx[:len(msg)] = msg
rx = bytearray(BUF_SIZE)

for count in range(1, 1000):
    transfer(spi, cs, tx, rx)

    # Parse the received data (skip NULL & non-printable)
    end = rx.find(b'\x00')
    data = bytes(rx[:end]) if end != -1 else bytes(rx)
    rx_str = ''.join(chr(b) for b in data if 32 <= b < 127)

    print("#%d TX: %s" % (count, msg.decode()))
    print("#%d RX: %s" % (count, rx_str))
    print()

    time.sleep(2)
```

6. Operation Procedure

Why use two environments? MicroPython's `machine.SPI` module only supports SPI master mode and lacks a slave API. Therefore, this experiment uses MicroPython + Thonny as the master and Arduino + Arduino IDE as the slave (implemented through the ESP-IDF's `spi_slave` driver). The two ESP32-S3s are connected via five DuPont wires.

6.1 Arduino IDE Environment Setup

Master Device (MicroPython): ESP32-S3 connected via Thonny IDE, refer to [MicroPython → 1. Before Development → 3. Thonny IDE](#)

Slave Device (Arduino): Compile and flash `slave.ino` using Arduino IDE. If ESP32-S3 board support is not yet installed, see [Arduino → 1. Before Development → 1. Environment Setup](#); flashing procedure is documented at [Arduino → 1. Before Development → 2. Build and Flash](#)

6.2 Flashing and Running

1. Connect the master and slave boards according to the wiring diagram in 3.2 (four wires + GND)
2. **Power on the slave first:** Open `slave.ino` with Arduino IDE, select the board `ESP32S3 Dev Module`, and flash and run
 - Slave source code location: `5.Source Code → MicroPython → 3.Serial Communication and Storage System → 3. SPI_Master → slave.ino`
3. **Run the master:** Open `master.py` with Thonny IDE and run
 - Master source code location: `5.Source Code → MicroPython → 3.Serial Communication and Storage System → 3. SPI_Master → master.py`

6.3 Arduino Slave Code (slave.ino)

The slave uses the ESP-IDF `spi_slave` API (directly callable in the Arduino environment), configured as SPI2_HOST + MODE0, and blocks waiting for the master to transfer. After receiving data, it prints it on the serial port and prepares the next reply based on the received content:

```
#include <driver/spi_slave.h>

#define PIN_SCK    4           // SPI clock input
#define PIN_MOSI   5           // Master Out, Slave In
#define PIN_MISO   6           // Slave Out, Master In
#define PIN_CS     10          // Chip Select input (active low)
#define BUF_SIZE   64          // Must match the master BUF_SIZE

void setup() {
    Serial.begin(115200);

    spi_bus_config_t bus_cfg = {};
    bus_cfg.mosi_io_num    = PIN_MOSI;
    bus_cfg.miso_io_num    = PIN_MISO;
    bus_cfg.sclk_io_num    = PIN_SCK;
    bus_cfg.quadwp_io_num  = -1;
    bus_cfg.quadhd_io_num  = -1;
    bus_cfg.max_transfer_sz = BUF_SIZE;

    spi_slave_interface_config_t slv_cfg = {};
    slv_cfg.mode            = 0;           // SPI_MODE0, must match the master
    slv_cfg.spics_io_num    = PIN_CS;
    slv_cfg.queue_size      = 1;

    spi_slave_initialize(SPI2_HOST, &bus_cfg, &slv_cfg, SPI_DMA_CH_AUTO);

    memset(tx_buf, 0, BUF_SIZE);
    strcpy((char*)tx_buf, "Slave ready, waiting..."); // First-round placeholder
}

void loop() {
    memset(rx_buf, 0, BUF_SIZE);

    spi_slave_transaction_t t = {};
    t.length    = BUF_SIZE * 8; // Transfer length in bits
    t.tx_buffer = tx_buf;       // Send buffer → MISO
```

```

t.rx_buffer = rx_buf;           // Receive buffer ← MOSI

spi_slave_transmit(SPI2_HOST, &t, pdMS_TO_TICKS(5000)); // Blocking wait, 5s
timeout

// Prepare the next reply from the received data
memset(tx_buf, 0, BUF_SIZE);
snprintf((char*)tx_buf, BUF_SIZE, "I got: %s", (char*)rx_buf);
}

```

Why can the slave call ESP-IDF functions? The Arduino-ESP32 framework is based on ESP-IDF. `#include <driver/spi_slave.h>` directly calls the IDF's SPI slave API, without needing to install any additional library.

6.4 SPI Full-Duplex Timing Description

SPI transmits and receives **simultaneously** — while the master sends the Nth round of data, the slave is also sending the **N-1th round** of prepared tx_buf on MISO. Therefore, in the first round the master does not receive `"I got: Hello from MASTER"`, but the first-round placeholder string filled at the slave's initialization:

```

Round 1:
  Master sends → "Hello from MASTER"
  Slave sends → "Slave ready, waiting..." ← First-round placeholder, tx_buf is
still what was filled at init
  Slave prepares tx_buf = "I got: Hello from MASTER"

Round 2:
  Master sends → "Hello from MASTER"
  Slave sends → "I got: Hello from MASTER" ← From round 2 onwards it is the
correct reply

```

This is normal SPI full-duplex behavior, not a bug. If you mind the first-round content, you can change the placeholder string of tx_buf in `setup()` to an empty string or directly write `"I got: "`.

6.5 Master Serial Output Example

```

=== SPI Master ===

[MASTER] SPI(1) initialized
        SCK=4 MOSI=5 MISO=6 CS=10
[MASTER] Starting...

[#1] TX: Hello from MASTER
[#1] RX: I got: Hello from MASTER

[#2] TX: Hello from MASTER
[#2] RX: I got: Hello from MASTER
...

```

```

=== SPI Master ===

[MASTER] SPI(1) initialized
        SCK=4 MOSI=5 MISO=6 CS=10
[MASTER] Starting...

[#1] TX: Hello from MASTER
[#1] RX: I got: Hello from MASTER

[#2] TX: Hello from MASTER
[#2] RX: I got: Hello from MASTER

```

6.6 Slave Serial Output Example

```

=== SPI SLAVE ===
pins: SCK=4 MOSI=5 MISO=6 CS=10
Ready.
--- [SLAVE] ---
    bytes=64  strlen=17
    RX: [Hello from MASTER]
--- [SLAVE] ---
    bytes=64  strlen=17
    RX: [Hello from MASTER]
...

```

```

=== SPI SLAVE ===
pins: SCK=4 MOSI=5 MISO=6 CS=10
Ready.
--- [SLAVE] ---
    bytes=64  strlen=17
    RX: [Hello from MASTER]
--- [SLAVE] ---
    bytes=64  strlen=17
    RX: [Hello from MASTER]

```

6.7 Verification Method

Action	Expected result
Slave powered on, master running	Round 1 RX is the placeholder string; from round 2 RX= "I got: Hello from MASTER"
Slave powered off	Master RX is all empty (MISO high-impedance)
Master changes msg=b"TEST"	Slave RX becomes "TEST", master RX= "I got: TEST"
Remove any wire	Communication breaks, slave TIMEOUT

7. Common Issues

Symptom	Cause	Solution
RX is all 0x00 or empty	Slave not connected or CS not pulled low	Check the four-wire wiring, confirm the slave has started
Slave prints <code>TIMEOUT (263)</code>	CS pin conflict, e.g. GPIO2 occupied by the on-board LED	Use GPIO10 instead (avoid strapping pins and on-board peripherals)
Round 1 RX is the placeholder string instead of <code>"I got: ..."</code>	Normal SPI full-duplex behavior: the slave sends the tx_buf prepared in the previous round	Normal; it is correct from round 2 onwards
Data misalignment/garbage	SPI mode mismatch	Confirm polarity and phase match on both the master and slave
SPI initialization fails	Pin conflict	Avoid occupied GPIOs (e.g. flash pins 26~32)
Master and slave pins do not correspond	No RX data	Confirm the master and slave GPIOs correspond one to one (e.g. GPIO4↔GPIO4, CS=GPIO10↔GPIO10)